Fan-out as a design variable in an extensible index framework

Andrey Borodin

Аннотация

The height of a tree index and the cost of every descent through it are governed by node fan-out, and fan-out is governed by how many bytes an entry occupies. In an extensible framework, where the key type and its operations are supplied from outside the core, several bytes per entry are spent on nothing: on representations that duplicate the indexed value, on attributes that navigation never consults, and on rewriting entries that have not moved. This paper reports three changes that recover them, their implementation in PostgreSQL, and what they cost.

The first is an interface result rather than an algorithmic one. The framework required every operator class to supply a pair of functions converting between the indexed value and its stored form, even when the two coincide and the functions were identities. Making them optional removes the identity call performed once per entry of every visited node; more usefully, it turns the *absence* of a function into information the core can act on. Knowing that the stored form is the original value, the core can answer a query from the index alone, without a further function that previously had to be supplied for exactly this purpose. The second separates key attributes from covering attributes, so that columns needed only in the answer are stored in leaves and never widen the keys that determine fan-out at every internal level; unlike the corresponding B-tree feature, the motivating case here is a column whose type has no operator class at all. The third replaces deletion-and-reinsertion of an entry by overwriting it in place, which for the common case of a key widened during descent moves no data at all.

All three have been in released versions since PostgreSQL 10–14. We report the fan-out model they act on, the measurements, and one consequence that surfaced during review: an optimisation of page layout changes the bytes a page contains after log replay, which matters to tooling that compares pages byte for byte.

1 Introduction

A tree index answers a query by descending from the root, and the cost of that descent has two parts: the number of nodes visited, and the work done inside each. Both are governed by fan-out. Fan-out sets the height of the tree, so it sets how many nodes a descent visits; and it sets how many entries a visited node contains, so it sets the work done there.

Fan-out, in turn, is arithmetic on bytes:

$$f = \left\lfloor \frac{B - h}{k + p + \sigma} \right\rfloor, \tag{1}$$

for page size B, page header h, key size k, downlink size p and per-entry overhead σ. Every byte an entry does not need is paid for at every level of every descent, for the life of the index.

In an *extensible* framework the accounting is worse than in a purpose-built index, because the core does not know what a key means and must therefore be conservative about it. This paper is about three places where that conservatism cost bytes or cycles for nothing, in a generalized search tree [2] as implemented in PostgreSQL.

The first is not an algorithm but an interface. The framework obliged every operator class to supply two functions converting between the indexed value and its stored representation, even when the two coincide and both functions were identities. Making them optional removes an identity call performed once per entry of every visited node. The more interesting consequence is that the absence of a function becomes information: if no conversion is supplied, the core knows the stored form *is* the original value,

and can answer a query from the index alone — something that previously required supplying yet another function.

The second separates the attributes a query needs to *answer* from those it needs to *navigate*. Storing the former only in leaves keeps them out of the keys that set fan-out at every internal level.

The third concerns entries that are modified rather than added. A descent widens the keys on its path; the original implementation performed each widening as a deletion followed by an append, moving the data of the page and destroying the correspondence between entry order and physical order.

Contributions.

1. Optional support functions for key conversion, and the reading of their absence as a statement about the stored representation, which yields index-only scans without an additional function (Section 3).
2. Covering attributes for a framework in which the covered column may have no operator class of its own, with the fan-out argument that distinguishes this case from the corresponding B-tree feature (Section 4).
3. In-place overwriting of an entry, which for the common case of a key widened during descent moves no data (Section 5), and an account of the compatibility consequence it surfaced (Section 6).

2 Background

A generalized search tree is a balanced tree whose node occupies one page and whose entries pair a key with a downlink, the key covering everything in the subtree below. The key type and its operations belong to an operator class supplied outside the core: `consistent` decides whether a subtree may hold an answer, `union` produces a key covering a set of keys, `penalty` prices the widening of a key, `picksplit` partitions an overflowing node. Substituting bounding rectangles yields an R-tree, signatures a full-text index, ranges a range index.

Two further functions concern representation rather than search. `compress` converted an indexed value into the form stored in the tree, `decompress` converted it back. The intent was to store something smaller than the value — a bounding rectangle in place of a polygon. A third, `fetch`, reconstructs the original value from the stored form and exists so that a query can be answered without visiting the table.

Height follows from fan-out as $H \approx \log_f N$, so (1) enters the cost of a point query as $\sum_l \omega_l(c_r + fc_c)$ — a page access per visited node plus a per-entry cost inside it. Fan-out therefore acts in two directions at once: it reduces the number of nodes visited and increases the work in each. The changes below all move k or σ, that is the denominator of (1).

3 The absence of a function as information

3.1 The problem

For a substantial share of operator classes no conversion is wanted: the stored representation is the indexed value. The framework nevertheless required both functions, so such classes defined identities. Beyond the redundant code this had two effects.

The core could not tell whether the stored form equalled the original value. Lacking that knowledge it could not answer a query from the index alone, and an operator class wanting index-only scans had to supply `fetch` — a third function, whose body for these classes is again an identity.

And the identity `decompress` was called for every entry of every scanned page, that is $O(f)$ times per visited node. In terms of Section 2 this is a contribution to c_c, the per-entry cost, which by (1) grows as fan-out improves. An identity function call is cheap; multiplied by fan-out and by tree height and by every query, it is not free, and it is pure overhead.

3.2 The change

Both functions were made optional, and their absence given meaning. When no `compress` is defined, the core knows the stored representation is the indexed value itself, and therefore

- answers queries from the index alone without requiring `fetch`; and
- performs no conversion when scanning a node.

Implemented in `d3a4f89d8a3`, released in PostgreSQL 11.

3.3 Why this is worth reporting as a result

The performance part is a small constant. The part we consider more useful is the interface part, and it generalizes past this framework.

An extensible system defines what an extension must provide. The obligation is usually read as a floor: here is the minimum you must implement. But every mandatory element also destroys information, because a requirement satisfied by everyone distinguishes nobody. When identity conversions are mandatory, the class that needs no conversion is indistinguishable from the class that needs one, and the core must assume the harder case for all. Making the element optional restores the distinction and lets the core specialize.

This is why the change removes an obligation and *adds* a capability at the same time, which is otherwise a surprising combination. The cost of a mandatory interface element is not only the effort of implementing it; it is the loss of whatever could have been inferred from choosing not to.

There is a practical corollary for extensibility, which is the property this framework exists to provide. The barrier to writing a new operator class is set by the volume of obligatory scaffolding, and two of the required functions here were scaffolding for the majority of classes.

4 Covering attributes

4.1 The problem

A query can be answered from the index alone when every column it needs is in the index. The naive route is to add the columns to the key. For a generalized search tree this fails twice.

First, a key attribute needs an operator class for its type, and the column one wants in the answer may have none — there is no meaningful notion of a bounding key for it, and no reason there should be. Second, and quantitatively, a key attribute widens k at *every* level. By (1) this lowers fan-out at every internal level, and hence raises H; a column that navigation never consults would make every descent longer.

4.2 The change

Index attributes were divided into key and covering. Covering attributes are stored only in leaf pages, take no part in navigation, do not enter the keys of internal nodes, and require no operator class for their type. Fan-out at internal levels is then given by (1) with k over the key attributes alone, and at the leaf level with $k + k_{\text{inc}}$. Since height depends on internal fan-out, covering attributes do not increase it.

Implemented in `f2e403803fe`, released in PostgreSQL 12. The corresponding change for the space-partitioned tree — `09c1c6ab4bc`, released in 14 — is the work of Pavel Borisov, and we mention it because it shows the argument transferring to a second structure, not because it is ours.

4.3 How this differs from the B-tree case

The feature has a B-tree namesake and a different motivation. There, covering attributes serve mainly to place a uniqueness constraint on part of an index's attributes while keeping other columns available in the index; the type of a covered column is not the difficulty, since a B-tree operator class exists for essentially every scalar type one would index.

Here the primary case is the column whose type has *no* generalized search tree operator class, and could not sensibly have one. A spatial index on a geometry column that must also return a name or an identifier is not an exotic requirement, and before this change it forced a visit to the table for every matching row. The two features share a syntax and solve different problems, which is worth stating because the shared name invites the assumption that the reasoning transfers.

5 Modifying an entry in place

Insertion widens keys along its path: the parent key must cover the newly inserted entry. Each widening modifies an entry already on a page.

The original implementation performed the modification as a deletion followed by an append. Entries are stored compactly, so deletion moves all data below the removed entry, and the modified entry lands at the end of the page. The second effect is the less obvious one: entry order and physical order stop corresponding, which costs processor cache locality on every subsequent scan of that page — again a contribution to c_c.

The change introduces overwriting in place. When the new entry has the size of the old one, which is the typical case for a widened key, no data moves at all; when the sizes differ, only the difference moves. Entry order is preserved.

Implemented in `b1328d78f88`, released in PostgreSQL 10. This result was published previously [1], with measurements against release 10 showing insertion of sorted data about three times faster and about 15 % on random data, alongside a smaller index; we restate it here for completeness of the fan-out picture rather than as a contribution of this paper, and Section 7 re-measures it on a current release.

6 A page-layout change is a compatibility change

One consequence surfaced during review and deserves its own section, because it is not specific to this optimisation.

After the change, replaying the write-ahead log produces a page whose bytes differ from those produced before, although the page is logically equivalent: the same entries, in the same order, at different offsets. Nothing in the database's own operation depends on the difference. But tools that verify replay by comparing a replica's page against the primary's byte for byte do depend on it, and such tools are how one gains confidence that replay is correct.

The general form: an optimisation of page layout is a change to an interface that is not written down anywhere. The set of byte-level representations a page may legitimately have is a contract with everything outside the database that reads pages, and it has no declaration to consult. We note it here as a hazard for anyone proposing a similar change, and as an argument for the structural verification approach we develop elsewhere: comparing pages byte for byte is a brittle proxy for the property actually wanted, which is that the two pages carry the same entries.

7 Evaluation

7.1 Optional support functions: a capability, not a speed

The claim of Section 3 is binary, and so is its test. We define an operator class for `box` — a type whose stored representation is its input representation — omitting both the conversion function and the fetch

Таблица 1: Three ways to make a column available to a query

column is	index pages	internal pages	internal fan-out	accesses/query
absent	8 160	66	122.6	4.19
covering	9 622	77	123.9	4.46
in the key	9 638	93	102.6	4.61

Таблица 2: Insertion time, 500 000 rows, medians of three runs

key	build	sorted input, s	random input, s
`point`, 16 B	before	9.73	7.15
	after	9.74	7.06
`cube`, 256 B	before	29.96	18.87
	after	**27.17**	19.26

function, and try to use it.

build	outcome
before `d3a4f89d8a3`	index cannot be built: *missing support function 3*
after	index builds; index-only scan is available

Both halves of the claim are established by the same experiment. The class becomes usable, and the query is answered from the index alone although no fetch function was supplied — which is the capability that the absence of a conversion function is what tells the core about.

We deliberately report no timing here. The author of the change stated at the time that the speed benefit is small and that the point is the reduction of obligatory boilerplate, and we have no reason to dress that up.

7.2 Covering attributes

One million points with an additional integer column, 2000 range queries, PostgreSQL 18.4, with the extension that supplies a generalized search tree operator class for integers installed — without it the third row below is not merely worse but impossible.

The prediction of Section 4 is confirmed in exactly the quantity it was made about. A covering attribute leaves internal fan-out untouched, 122.6 against 123.9; putting the same column in the key lowers it to 102.6, a loss of 16 %, and raises the number of internal pages from 77 to 93. The leaf level is identical in the two cases — 9 544 pages either way — so the whole difference falls on the navigational part of the tree, which is what (1) says it should.

It is worth stating the categorical part before the numbers, because it is the stronger claim. Without a third-party extension the index with the column in the key cannot be created at all: the core supplies no operator class for the integer type. The naive route is not an inferior option in general; it is unavailable unless someone has separately supplied the missing class.

7.3 In-place overwrite: the effect is real and smaller than published

Measured first on `point` keys, this change made no difference at all. That is consistent with its mechanism — a 16-byte entry has almost nothing to move when overwritten — so we repeated it with `cube` keys of dimension 16, that is 256 bytes.

With a large key the effect appears and is reproducible: 1.10 on sorted input, with runs within 0.05 seconds of each other. On random input there is no difference at any key size, which matches the mechanism: sorted insertion widens a parent key at nearly every step, random insertion much more rarely.

The previously published figures for this change [1] are about threefold on sorted input and a smaller index. We measure 10 % and an index 2 % larger. The gap is too wide to attribute to conditions, and we have not identified its cause; the 2017 measurement may have used a different workload, a different operator class, or a build with assertions enabled. We therefore report what we measured, and treat the earlier figures as belonging to their own setting rather than as re-established here. The direction of the mechanism — benefit proportional to the volume of data moved, hence to key size — is confirmed.

8 Limitations

The three changes act on different terms and are not interchangeable: the first and third reduce c_c, the second protects k at internal levels, and none of them affects the overlap of keys, which for a multidimensional index dominates the cost of a selective query. Fan-out is a lever with a limit: raising it reduces the number of nodes visited but increases the work inside each, so a larger page is not uniformly better, and past some point only an in-page structure that avoids examining all f entries can help further.

The interface argument of Section 3 rests on one framework and one pair of functions. We believe it generalizes — a mandatory element that everyone satisfies identically carries no information — but we have one instance of it, not a survey.

9 Conclusion

Fan-out is usually treated as a property of the data: keys are the size they are. In an extensible framework a noticeable part of it is instead a property of the interface between the core and the code that supplies key semantics, and can be recovered by changing that interface rather than the data.

Of the three changes reported here, the one we would highlight is the smallest in its performance effect. Making two support functions optional removes an obligation from every operator class author and simultaneously grants the core a capability it did not have, because a requirement that everyone satisfies identically conveys nothing, and dropping it lets the core learn something from the choice not to implement. That reading — optionality as a channel of information rather than as a convenience — seems to us the transferable part of this work.

Acknowledgements

Список литературы

[1] Andrey Borodin, Sergey Mirvoda, Ilia Kulikov, and Sergey Porshnev. Optimization of memory operations in generalized search trees of postgresql. In *Beyond Databases, Architectures and Structures (BDAS)*, Communications in Computer and Information Science, pages 224–232. Springer, 2017.

[2] Joseph M Hellerstein, Jeffrey F Naughton, and Avi Pfeffer. *Generalized search trees for database systems*. University of Wisconsin-Madison, 1995.